AWK

April 19, 2026

Canine-Table

POSIX Nexus serves as a comprehensive cross-language reference hub that explores the implementation and behavior of POSIX-compliant functionality across a diverse set of programming environments. Built atop the foundational IEEE Portable Operating System Interface (POSIX) standards, this project emphasizes compatibility, portability, and interoperability between operating systems.

Abstract

Contents

I I/O Modules

I File Readability Sentinel

nx_is_file Arguments

- </> D ⇝ Candidate file path to probe
- </> B ⇝ Boolean or mode flag controlling strictness of the readability test

```
1 function nx_is_file(D, B)
```

File Readability Sentinel — The Probe Glyph

- **Purpose** ⇝ Determine whether a given path ‘D’ refers to a readable file, with strict or permissive content requirements depending on ‘B’
- **Input** ⇝ ‘D’ is the file to test; ‘B’ selects the probe threshold:
  - </> $B = ""$ ⇝ Strict mode: `getline < D` must return > **0**, meaning the file is readable *and contains at least one line of content*
  - </> $B \neq ""$ ⇝ Permissive mode: `getline < D` must return > **-1**, meaning the file is readable even if it is *empty*
- **Mechanism** ⇝
  - Empty path/If ‘D’ is empty, immediately return ‘0’
  - Probe/Attempt `getline < D` using the threshold determined by ‘B’
  - Close/If the probe succeeds, close the stream handle
  - Failure/If the probe fails the threshold, return ‘0’
  - Success/Otherwise return ‘1’
- **Output** ⇝ Returns ‘1’ if the file is readable under the chosen mode, ‘0’ otherwise
- **Use Case** ⇝ Foundational predicate for all file-identity and include-expansion rituals; ensures that only real, readable files (optionally non-empty) enter the semantic universe

I Examples

Strict readability check (non-empty file required)

- State ⇝ D=‘`config.cfg`’, B=‘ ’ (strict mode)
- Operation ⇝ Probe succeeds only if ‘config.cfg’ is readable *and contains at least one line*
- Expected Result ⇝ Returns ‘1’ for non-empty readable files; ‘0’ for empty or unreadable files

```
nx_is_file("config.cfg", " ")
```

Permissive readability check (empty file allowed)

- State ⇝ D=‘`empty.log`’, B=‘1’ (permissive mode)
- Operation ⇝ Probe succeeds if ‘empty.log’ is readable, even if it contains no content
- Expected Result ⇝ Returns ‘1’ for readable files regardless of content; ‘0’ only if unreadable

```
nx_is_file("empty.log", 1)
```

I Path Normalization Conductor

nx_file_path Arguments

- D1 ⇝ Input path to normalize or dissect
- B ⇝ Boolean or mode flag selecting which component to return
- D2 ⇝ Directory separator (default “/”)
- V ⇝ Prefix-expansion map for ‘NX_*:/’, ‘~/’, and ‘-/’

```
function nx_file_path(D1, B, D2, V)
```

Path Normalization Conductor — The Dissection Glyph

- **Purpose** — Normalize a file path, expand prefix sigils, collapse redundant separators, and optionally extract basename or directory components depending on ‘B’
- **Input** — ‘D1’ is the raw path; ‘B’ selects the return mode; ‘D2’ is the directory separator; ‘V’ holds prefix expansions:
  - </> $B = ""$ — Return the *basename* (final component)
  - </> $B = 0$ — Return the *full normalized path*
  - </> $B \neq ""$ — Return the *directory path* (parent directory)
- **Mechanism**
  - Prefix expansion/If ‘D1’ begins with ‘-/’, ‘~/’, or ‘NX_*:/’, expand using ‘V’ (environment-derived prefix map)
  - Separator collapse/Reduce repeated separators ‘D2+’ to a single ‘D2’
  - Trailing cleanup/Remove trailing separators unless the path is root
  - Basename extraction/If $B = ""$, strip directory prefix and return final component
  - Directory extraction/If $B \neq ""$, strip basename and return parent directory
  - Full path/If $B = 0$, return the normalized path unchanged
- **Output** — A normalized path, basename, or directory path depending on ‘B’
- **Use Case** — Core utility for all file-identity and include-expansion rituals; ensures consistent path semantics across prefix expansions, directory sweeps, and file lookups

I Examples

Extract basename (strict basename mode)

- State → D1=‘/usr/local/bin/awk’, B=‘’
- Operation → Return only the final component
- Expected Result → ‘awk’

```
nx_file_path("/usr/local/bin/awk", "", "/")
```

Return full normalized path

- State → D1=‘ /projects///nx’, B=0
- Operation → Expand prefix, collapse separators, return full path
- Expected Result → Normalized absolute path to ‘nx’

```
nx_file_path("~/projects///nx", 0, "/", V)
```

Extract directory path (parent directory)

- State → D1=‘NX_C:/config/main.cfg’, B=1
- Operation → Expand ‘NX_C:/’, strip basename, return directory
- Expected Result → Parent directory of ‘main.cfg’

```
nx_file_path("NX_C:/config/main.cfg", 1, "/", V)
```

I Extension Sweep Conductor

nx_file_path_expand Arguments

- D1 — Primary prefix path used when constructing candidate files
- D2 — Secondary prefix path used for alternate construction
- D3 — Basename to sweep for extensions
- D4 — Extension-separator regex (default ‘[.]’)
- D5 — Directory separator (default “/”)
- V1 — Parallel-array registry for file identity triples
- V2 — Prefix-expansion map for ‘NX_*:/’, ‘~/’, and ‘-/’
- B1 — Boolean/mode flag selecting sweep strategy
- B2 — Boolean/mode flag passed to `nx_is_file`

```
function nx_file_path_expand(D1, D2, D3, D4, D5, V1, V2, B1, B2)
```

Extension Sweep Conductor — The Expansion Glyph

- **Purpose** — Perform an extension-sweep over a basename ‘D3’, generating candidate paths by peeling extensions and testing each candidate with `nx_file_store`
- **Input** — ‘D1’ and ‘D2’ are prefix paths; ‘D3’ is the basename; ‘D4’ and ‘D5’ define separators; ‘V1’ and ‘V2’ support file resolution; ‘B1’ and ‘B2’ control sweep and readability modes:
  - *B1* — Sweep mode: determines whether extension-peeling is active
  - *B2* — Readability mode passed to `nx_is_file`
- **Mechanism**
  - Reverse scan/Reverse ‘D3’ to locate extension boundaries using ‘D4’
  - Peel extension/Each match shortens the basename and accumulates peeled extensions into ‘ext’
  - Construct candidates/For each peel, construct:
    - **Candidate 1** — ‘D1 ext D5 bs’
    - **Candidate 2** — ‘D2 ext D5 D3’
  - Probe/Call `nx_file_store` on each candidate; success halts the sweep
  - Termination/Stop when a readable file is found or no more extension boundaries remain
- **Output** — Returns ‘1’ if any candidate path resolves to a readable file; ‘-1’ otherwise
- **Use Case** — Used by `nx_file_store` when fallback mode requests extension-peeling; enables discovery of files such as ‘config.json’, ‘config.yaml’, ‘config.conf’ by sweeping suffixes of ‘config’

I Examples

Sweep for alternate extensions

- **State** ⇝ Basename ‘config.json’, prefixes ‘D1=‘/etc/’’, ‘D2=‘/usr/local/etc/’’
- **Operation** ⇝ Peel ‘.json’, test ‘config’, then test ‘config.json’ under both prefixes
- **Expected Result** ⇝ Returns ‘1’ if any candidate resolves to a readable file

```
nx_file_path_expand("/etc/", "/usr/local/etc/", "config.json", "[.]", "/", V1, V2, 1, 1)
```

Multiple extension layers (e.g., ‘.tar.gz’)

- **State** ⇝ Basename ‘archive.tar.gz’
- **Operation** ⇝ Peel ‘.gz’, then ‘.tar’, constructing candidates at each stage
- **Expected Result** ⇝ First readable candidate halts the sweep and returns ‘1’

```
nx_file_path_expand("/data/", "/backup/", "archive.tar.gz", "[.]", "/", V1, V2, 1, 1)
```

I File Identity Conductor

nx_file_store Arguments

- V1 ⟿ Parallel-array registry storing file identity triples
- D1 ⟿ Input file name (basename or relative path)
- D2 ⟿ Optional prefix path to search under
- V2 ⟿ Prefix-expansion map for ‘NX_*:/’, ‘~/’, and ‘-/’
- D3 ⟿ Directory separator (default “/”)
- B1 ⟿ Boolean/mode flag selecting fallback strategy
- B2 ⟿ Boolean/mode flag passed to `nx_is_file`
- D4 ⟿ Extension-separator regex used by `nx_file_path_expand`

```
function nx_file_store(V1, D1, D2, V2, D3, B1, B2, D4)
```

File Identity Conductor — The Triple Glyph

- **Purpose** ⟿ Resolve a file path, verify readability, and register its identity triple (real path, directory path, basename) into a bijective parallel-array registry
- **Input** ⟿ ‘D1’ is the candidate file; ‘D2’ is an optional search prefix; ‘V2’ holds prefix expansions; ‘B1’ and ‘B2’ control fallback and readability modes:
  - </> $B1 = 0$ ⟿ No fallback: only direct path resolution attempted
  - </> $B1 = 1$ ⟿ Extension-sweep fallback via `nx_file_path_expand`
  - </> $B1 = 2$ ⟿ Directory-sweep fallback across known directories in ‘V1’
  - </> $B1 = 3$ ⟿ Combined sweep: extension first, then directory
  - </> $B2$ ⟿ Strict/permissive readability mode passed to `nx_is_file`
- **Mechanism** ⟿
  - Normalize paths/Compute:
    - </> **rlp** ⟿ Resolved lookup path: ‘D2 + D1’ normalized
    - </> **orlp** ⟿ Original path: ‘D1’ normalized
    - </> **bs** ⟿ Basename extracted from ‘D1’
  - Direct probe/Test ‘rlp’ and ‘orlp’ with `nx_is_file` under mode ‘B2’
  - Directory sweep/If ‘B1 = 2’, iterate directories stored in ‘V1’ to locate a readable file
  - Extension sweep/If ‘B1 = 1’ or ‘3’, call `nx_file_path_expand` to peel extensions and test candidates
  - Identity triple/Once a readable file is found:
    - </> **Real Path** ⟿ Canonical resolved path
    - </> **Directory Path** ⟿ Parent directory
    - </> **Basename** ⟿ Final component of the file name
  - Registry insertion/Insert each component into ‘V1’ using `nx_bijective` and `nx_parr_stk`
  - Return code/‘1’ if newly registered, ‘0’ if already known, ‘-1’ on failure

Output ⟿ Returns ‘1’ for a new identity triple, ‘0’ if already present, ‘-1’ if no readable file could be resolved

Use Case ⟿ Foundation of all file-merge, overlay, and include-expansion systems; ensures every file entering the semantic universe has a stable, bijective identity triple

I Examples

Direct resolution (no fallback)

State ⟿ D1=‘config’, D2=‘NX_C:/’, B1=0

Operation ⟿ Resolve ‘NX_C:/config’ and register its identity triple

Expected Result ⟿ ‘1’ if new, ‘0’ if already known

```
nx_file_store(V1, "config", "NX_C:/", V2, "/", 0, "", "[.]")
```

Extension-sweep fallback

State ⟿ D1=‘theme’, B1=1

Operation ⟿ Try ‘theme’, then ‘theme.*’ by peeling extensions

Expected Result ⟿ First readable candidate registered

```
nx_file_store(V1, "theme", "", V2, "/", 1, "", "[.]")
```

Directory-sweep fallback

State ⟿ D1=‘settings.cfg’, B1=2

Operation ⟿ Search directories already registered in ‘V1’

Expected Result ⟿ First readable match registered

```
nx_file_store(V1, "settings.cfg", "", V2, "/", 2, "", "[.]")
```

I Include Directive Conductor

nx_file_merge Arguments

- D1 ⟿ Root file name or input stream to expand
- D2 ⟿ File-exclusion list; omit these if encountered after a directive
- D3 ⟿ Separator used to split the delimiter specification ‘D4’
- D4 ⟿ Five-field delimiter specification:
  - [1] ⟿ File-exclusion separator
  - [2] ⟿ Directive sigil
  - [3] ⟿ Directory separator
  - [4] ⟿ File-extension separator
  - [5] ⟿ Directive name
- B ⟿ Boolean/mode flag passed to `nx_file_store`
- N ⟿ Verbosity and diagnostic level

```
function nx_file_merge(D1, D2, D3, D4, B, N)
```

Include Directive Conductor — The Expansion Glyph

- **Purpose** — Traverse a root file, detect include directives, recursively load referenced files, and emit a fully expanded DFS-ordered text stream with cycle prevention and customizable directive syntax
- **Input** — 'D1' is the entry file; 'D2' lists files to omit; 'D3' and 'D4' define directive and path delimiters; 'B' controls fallback behavior in `nx_file_store`; 'N' controls diagnostics
- **Mechanism**
  - Parse delimiter specification/Split 'D4' using 'D3'; extract file-exclusion separator, directive sigil, directory separator, extension separator, and directive name; warn if more than five fields are provided
  - Validate delimiters/Each delimiter is checked via `nx_delim_sep` to ensure it is safe and unambiguous
  - Load root file/Resolve and register the root file using `nx_file_store`; abort if unreadable
  - Apply exclusion list/Split 'D2' using the exclusion separator; resolve each excluded file and update the drop-prefix to prevent re-inclusion
  - Construct directive regex/Build patterns matching the directive sigil, directive name, and filename argument
  - Recursive expansion/'stk[]' accumulates text fragments; 'trk[]' tracks pending include files; each include spawns a new DFS branch
  - Directive handling/If a line matches the directive:
    - **Prefix** — Extract text before the directive
    - **Argument** — Extract filename argument and attempt resolution via `nx_file_store`
    - **Branch** — If new file, push prefix and queue file for DFS expansion; otherwise emit prefix only
  - Non-directive lines/Preserved verbatim unless empty or whitespace-only
  - DFS flattening/After all recursive expansions, `nx_dfs(stk)` linearizes the include tree into a stable output order
  - Emission/Final merged text is printed in DFS order, preserving all non-empty fragments

Output Prints a fully expanded text stream with all include directives resolved; returns '-1' on delimiter or file-resolution failure

Use Case Core of the Nx preprocessor: enables modular documents, chained configuration files, recursive overlays, and controlled include semantics with cycle prevention and customizable syntax

I Examples

Basic include expansion

State D1='main.txt', directive sigil '#', directive name 'nx_include'

Operation Expand all '#nx_include file' directives recursively

Expected Result Merged output printed with all includes resolved in DFS order

```
nx_file_merge('main.txt', "", ",", "#,/,.,nx_include', 1, 1)
```

Exclude specific files during expansion

State D2='secret.cfg,debug.cfg'

Operation If encountered after a directive, these files are ignored

Expected Result Expansion proceeds without merging excluded files

```
nx_file_merge('main.txt', "secret.cfg,debug.cfg", ",",
↪"#,/,.,nx_include", 1, 1)
```